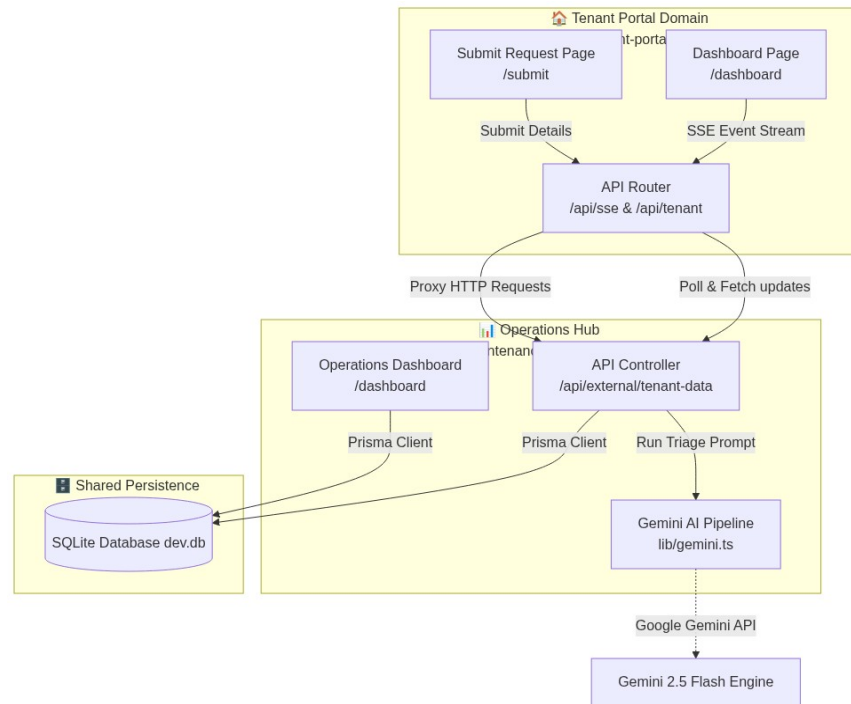


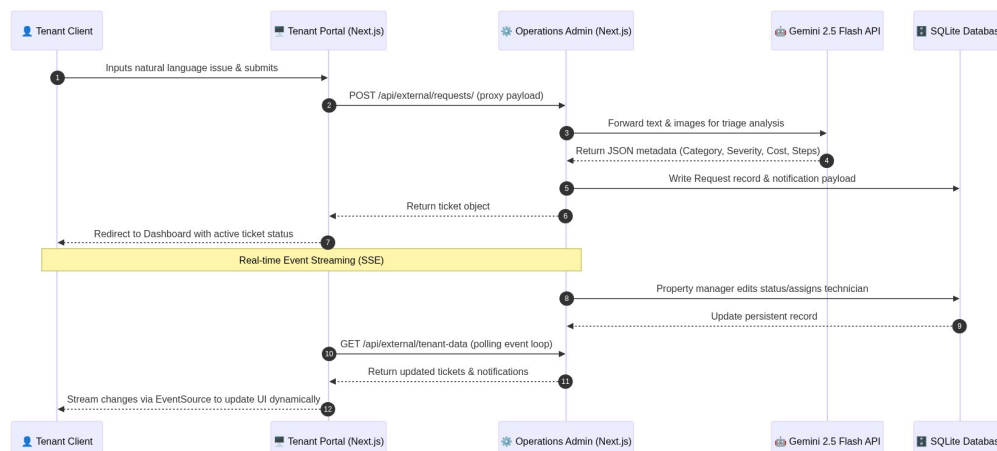
1. System Topology & High-Level Architecture

The platform uses a split-responsibility dual-application topology. This topology ensures administrative features and data mutation controls reside on a protected backend container, while tenants access a streamlined portal. Both apps interact with a single SQLite database running in WAL (Write-Ahead Logging) mode, resolving data conflicts through dedicated secure REST endpoints.



1.2 Request Flow Sequence Diagram

This sequence diagram outlines how requests are parsed by AI, saved, and synced in real-time between tenant and manager views.



2. Technology Stack & Core Directories

Both applications are built with modern web technologies:

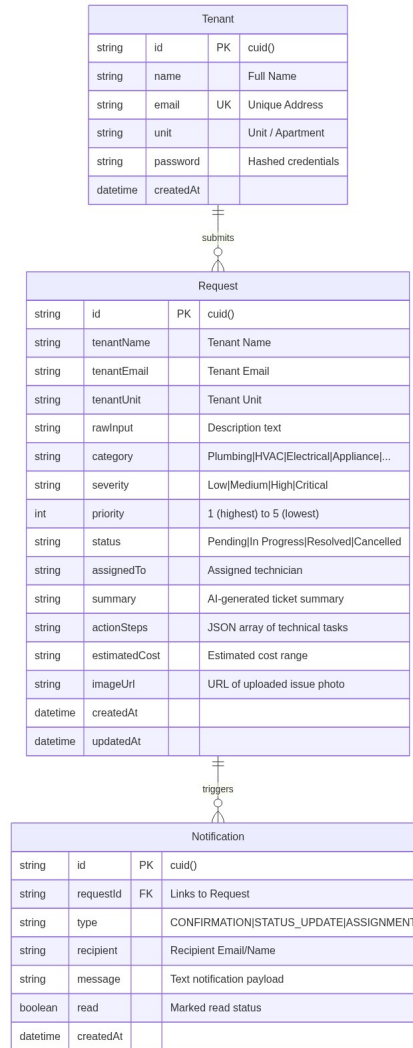
- **Frontend UI & Framework:** Next.js 16 (App Router), React 19, and Tailwind CSS.
- **Design System & Brand Identity:** Custom CSS variables matching the Adeer Navy & Teal theme, using custom Poppins typography, rounded-2xl glassmorphism, pulse loading animations, and responsive layouts.
- **Database ORM:** Prisma Client v7 querying a local SQLite datastore inside WAL mode to prevent locking during concurrent reads/writes.
- **AI Engine:** Google Gemini 2.5 Flash API for automated categorization, cost assessment, prioritization, and technician task checklist generation.

2.1 Key File Mapping

Component / Path	Description
<code>maintenance-app/prisma/schema.prisma</code>	Database schema definitions.
<code>maintenance-app/lib/gemini.ts</code>	Prompt engineering and JSON parser.
<code>maintenance-app/app/api/external/tenant-data/route.ts</code>	Cross-domain synchronizer API.
<code>tenant-app/app/api/sse/route.ts</code>	Server-Sent Events client-stream connector.
<code>tenant-app/app/dashboard/page.tsx</code>	Main tenant tracking dashboard workspace.
<code>tenant-app/app/page.tsx</code>	Dynamic authentication portal.

3. Database Schema Diagram

The SQLite database is managed using Prisma. The structure of the core tables is defined below:



4. Platform Interface Walkthrough

4.1 Tenant Portal Authentication

The Tenant Portal includes validation on both the client and server. If a tenant's session becomes invalid or expires, the system automatically redirects them to the login screen.

Captured at: 06/05/2026 04:54 PM URL: https://maintenance-app.fly.dev/submit

Portfolio Maintenance Demo Dashboard M. Hamid

Submit Maintenance Request

Describe your issue in your own words. Our AI will automatically classify, prioritize, and route your request.

Full Name * john.doe Email john@example.com

Unit / Apartment Number apt. 401-05

Attach a Photo (Optional)

Upload a file or drag and drop multiple files, up to 10MB (automatically compressed)

Describe the Issue *

Tell us what's wrong in your own words...

☒ Our AI will analyze your description to classify and prioritize the request.

Here are examples:

My kitchen sink is leaking water and it's flooding.

The hallway light on the 3rd floor has been flicking.

The heating system is not working and it's freezing.

There are ants coming from under the kitchen cabinet.

The front door lock is broken, I can't secure my door.

Submit Request

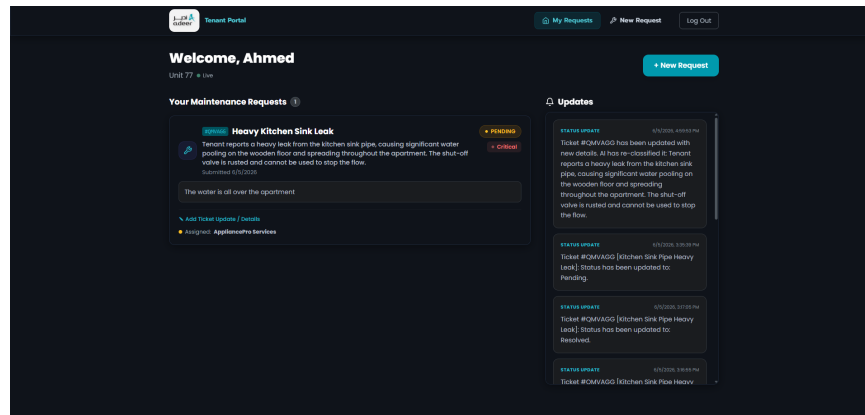
My Journey (About Me)

Designed & Developed by Dr. Mohamed Samir Hassan, MSc, PhD & AI and Innovation Manager, Portfolio & Maintenance Portal.

© 2026 All rights reserved. Powered by Digital Systems. All rights reserved.

4.2 Ticket Submission Form

The submit form features templates that allow tenants to quickly auto-populate common issue descriptions. The description is processed by the AI pipeline.



4.3 Automated AI Classification & Triage Pipeline

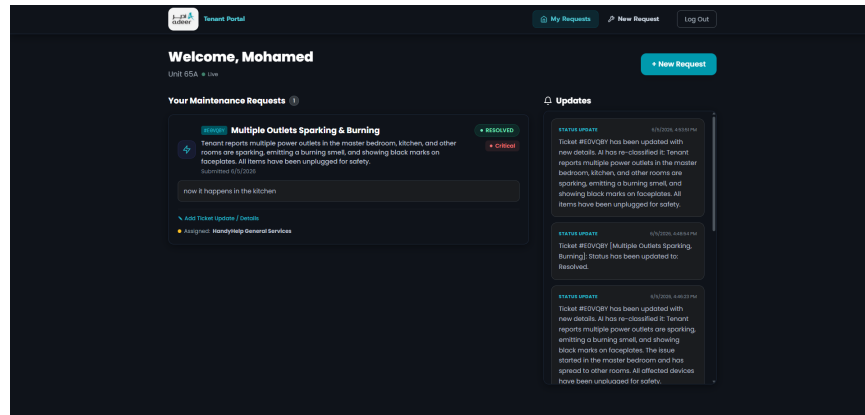
When a request is submitted, it is processed by the Gemini AI pipeline. The text description and any attached photos are analyzed to determine the ticket's category, urgency rating, estimated repair costs, and a structured set of repair steps.

```
const prompt = `You are an AI assistant for a property management company. Analyze the following maintenance request (and any attached images) from a tenant and provide a structured classification.`
```

Respond ONLY with a valid JSON object:

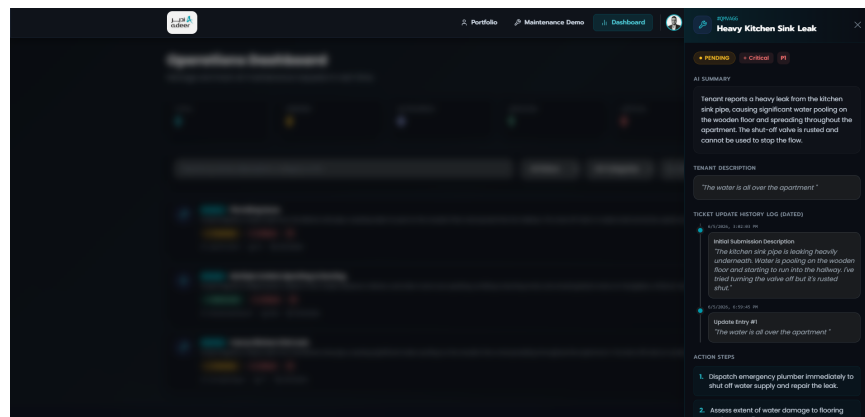
```
{
  "category": "Plumbing | Electrical | HVAC | Structural | Appliance | Pest Control | Cleaning | Security | Landscaping | General",
  "severity": "Low | Medium | High | Critical",
  "priority": "1 (highest) to 5 (lowest)",
  "summary": "Professional summary of the issue",
  "actionSteps": ["step 1", "step 2", "step 3"],
  "estimatedCost": "Estimated cost range"
};
```

The property manager sees this metadata inside the ticket sidebar:



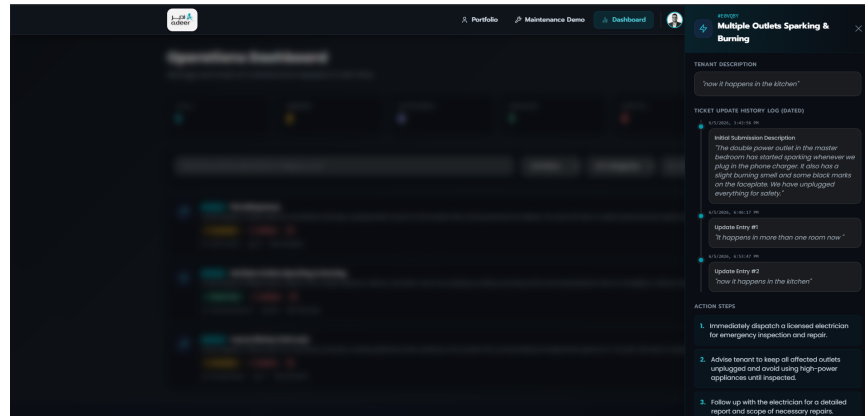
4.4 Operations Control Dashboard

The Operations Hub dashboard provides property managers with statistical counters, global search functionality, status filters, and priority tags to help manage incoming requests.



4.5 Live Client-Side Synchronization (SSE)

The Tenant Portal dashboard establishes a Server-Sent Events (SSE) connection to listen for updates. When a property manager updates a ticket's status, the change is streamed to the tenant's browser, updating the dashboard instantly without requiring a page reload.



5. Key Engineering Solutions & Fixes Implemented

- **EventSource Auto-Reconnection & Heartbeats:** To prevent routers and firewalls from dropping idle SSE streams, the server sends regular keep-alive pings. If a connection drops, the client automatically attempts to reconnect every 5 seconds, updating the status indicator to Reconnecting... during the downtime.
- **Cross-App State Sync:** To avoid data drift between the admin and tenant applications, the Tenant Portal fetches all ticket data directly from the admin database using secure REST calls. This configuration creates a single source of truth for the system's data.
- **Dynamic Image Scale & Pulsing Spinner:** Unified the loading screens across both applications to use a custom pulsing Adeer logo. Sizing class configurations use height-based scaling (h-5 w-auto) to preserve the logo's aspect ratio on different screen sizes.